

L2 image and filters

Resolution: sampling parameter, defined in dots per inch (DPI) or equivalent measures

RGB: linear **HSV**: nonlinear (Hue, Saturation, Value)

noise: salt and pepper, impulse, gaussian

boundary: 0, replicate, mirror

filter: gaussian (low pass; var $\leftrightarrow$ smooth), median (impulse/salt pepper, edge preserving)

convolution: flip the filter in both dim, cross-correlation; shift invariant; $f * g = g * f$;

$(f * g) * h = f * (g * h)$; $f * (g + h) = (f * g) + (f * h)$; $k f * g = f * k g = k (f * g)$; $f * e = f$; $F(f * g) = F(f) \cdot F(g)$

separability: rank=1 $\rightarrow$ separable; SVD;

template matching: matched $\rightarrow$ correlation map high value

L3 Camera Models

homogeneous coordinate: $ax + by + c = 0 \rightarrow I = (a, b, c)^T$; $I_1 \times I_2 \rightarrow I_3$, $I_3 \perp I_1, I_3 \perp I_2$

when pixel is not square: $f \rightarrow f_x, f_y$; $P = K[R|t]$; $P_l = [I|0]$, $P_r = [[e_r]_{\times} F|e_r]$

projection: many points to one; points to pts; lines to lines/point; planes to planes/lines
parallel lines converge at a vanishing point (vanishing point $\leftrightarrow$ direction)

Lines parallel to image plane are also parallel in the image; Lines on parallel 3D plane: the vanishing points lie on a vanishing line (horizon line for the ground plane)

all parallel lines same vanishing point: $Q = KX$, $X = V + tD$. Set $t \rightarrow \infty$, compute Q_x & Q_y .

World coord Q to camera coord: $Q' = R(Q - c)$, $RR^T = I$, $R[u \ v \ w] = I$; $Q' =$

$[R] - Rc][Q|1]^T$

L4 Stereo

parallel calibrated (known intrinsics and extrinsics) cameras: $Z = f \cdot T / (x_l - x_r)$

T : baseline, dist between optical center; $x_l - x_r$: disparity

Fundamental matrix F: $F \sim \lambda F$, $\det(F) = 0$ ($F_e = 0$) $\rightarrow$ use 7 pairs of points to get F

$l_r = Fx_l$, $e_r^T l_r = 0$, $e_r^T Fx_l = 0$ for any x_l , $e_r^T F = 0$ $p_r^T F p_l = 0$

$p_r = H_\pi p_l$, $l_r = e_r \times p_r = e_r \times H_\pi p_l = F p_l$, $F = [e_r]_{\times} \times H$ (invertible), $\text{rank}(F) = \text{rk}([e_r]_{\times}) = 2$

Structure from motion: bundle adjustment, min projected point err

L5 Keypoints

image features applications: tracking, camera pose estimation, instance matching, shot analysis (seg movie into different clips), scene understanding (complex global descriptor)

keypoints: locations; **descriptors**: features, content

recall ($TP / (TP + FN)$) $\leftarrow$ feat invariance **precision** ($TP / (TP + FP)$) $\leftarrow$ feat contrast and locality

SIFT descriptor (Scale Invariant Feature Transform)

1. build gaussian smoothing pyramid $L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$
2. compute $DoG = G(x, y, k\sigma) - G(x, y, \sigma) \approx (k - 1)\sigma \cdot \partial G / \partial \sigma$, $\partial G / \partial \sigma = \sigma \nabla^2 G$
3. find local extrema in DoG space ($3 \times 3 \times 3 - 1$ neighborhood) to detect keypoints
4. compute gradient, build orientation histogram, assign dominant orientation
5. 128 dim descriptor: 4×4 cells, each cell 8 orientation histogram, concat, normalize
6. Each keypoint = $(x, y, \sigma, \theta, 128 - D \text{ descriptor})$

Supervised Learning sol: learn from paired images

Self Supervised Learning sol (semantic preserving): DINOv2, CLIP/SigLIP, Stable Diffusion

harris corner detector: corner $\rightarrow$ significant change in all directions; not scale invariant

$f = \lambda_1 \lambda_2 / (\lambda_1 + \lambda_2)$, threshold $f > t$

$$E(u, v) = \sum_w w(x, y) [I(x + u, y + v) - I(x, y)]^2 = [u \ v] M \begin{bmatrix} u \\ v \end{bmatrix} \quad M = \sum_w w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

Keypoint matching: ratio = dist to best match / dist to sec best (low: good; high: ambiguous)

homogeneous coord trans matrix: affine: 2×3 ; projective: 3×3 , set scale = 1, 8 variables

mosaic(panorama): compute homography between images, transform, blend

RANSAC (RANDOM Sample Consensus): $1 - P = (1 - p^k)^S$ p : prob of any match valid; P : success

idea: correct line/transformation has many supporting points/matches

repeat N times: randomly draw a seed group (e.g. s points), fit line (least squares), find

inliners (e.g. $\text{dist} < t$), if # inliners $> d$, accept and refit to inliners

cons: tune params, work bad for low inliner ratios

L6 Vision NN

history: Mark I Perceptron, Adaline, backpropagation, Neocognitron, LeNet, ImageNet,

Receptive Field: $RF_l = RF_{l-1} + (k_l - 1) \cdot \prod_{i=1}^{l-1} s_i$

(calculation) input: $W_1 \times H_1 \times C$ - Number of filters K ; filter size F ; stride S ; zero padding P

conv output: $W_2 \times H_2 \times K$, $W_2 = (W_1 - F + 2P) / S + 1$; params: $F^2 C K$ weights and K biases

pooling output: $W_2 \times H_2 \times C$, $W_2 = (W_1 - F) / S + 1$, $H_2 = (H_1 - F) / S + 1$; num of params = 0

batchnorm output: $W_1 \times H_1 \times C$, params = $2C$ (each channel: γ_c, β_c) (running mean & running var not learnable) e.g. Input: $32 \times 32 \times 3$; 10 5×5 filters with stride 1, pad 2; Output: $(32 + 2 \times 2 - 5) / 1 + 1 = 32$ spatially, so $32 \times 32 \times 10$; num of params = $(5 \times 5 \times 3 + 1) \times 10$

Sigmoid: vanishing gradient; output not 0 centered; exp compute expensive

ReLU: output not 0 centered; gradient = 0 when $x < 0$ **ELU**: $\alpha(e^x - 1)$ when $x < 0$

Initialization: small random numbers $\rightarrow$ activations tend to 0 **Xavier init**: $W = N(0, 1/d_{in})$

prevent overfitting (reduce the gap between train and val): early stopping; data augmentation; add random noise during train, marginalize over noise during test (dropout, random crop, mixup, stochastic depth, data augmentation)

ResNet: conv+conv $\rightarrow$ res block, periodically double filters and downsize by stride 2;

Bottleneck Layer, $1 \times 1 \ 64 \rightarrow 3 \times 3 \ 64 \rightarrow 1 \times 1 \ 256$; Global Average Pooling, variable input size;

DenseNet: concat all layers before; **VGG**: replace large kernel by stacking small kernel;

ConvNeXt: Transformer ideas + CNN efficiency; Inverted bottleneck (channels++), GeLU, LN
Depthwise separable Conv: depthwise + pointwise conv; params: $K^2 C^2 \rightarrow K^2 C + C^2$ (img at end)

ViT: drop inductive bias and learn relationship; **U-Net**: see L9

self attention: great for long seq, highly parallel, $O(N^2)$ compute; flash att: $O(N)$ memory
masked/multiheaded self-attention (see image at the end of cheatsheet)

transformer: move layer norm before self attention and MLP; change LN to **RMS norm**

L7 Image Classification

identification: what is this landmark? **scene classification**: what type of scene?

classification: is this object a person/cat? **image annotation**: types of objects?

detection: where are all the cats? **segmentation**: which pixels belong to cats?

pose estimation: what is the pose of each object? give the line skeleton

attribute recognition: estimate attributes of the objects. E.g. red jacket, black hat

action recognition: what is happening? kneeling **surveillance**: why are they kneeling?

image classification challenge: viewpoint variation, background clutter, illumination, occlusion, deformation, intraclass variation

Self supervised learning: pretext objective -> downstream objective
 pretext obj: e.g. rotation prediction, image completion(inpainting), jigsaw, colorization
MAE: divide input into non-overlapping patches; mask a large proportion(75%, challenging&meaningful); do not send masked tokens into encoder; asymmetric design (large encoder – small decoder); positional embedding; cal loss on masked part
general pretext task: are these two pics related to the same object or not?
 contrastive learning: score (f(x),f(x+)) >> score(f(x),f(x-))
SimCLR: A Simple Framework for Contrastive Learning;
 x --f--> h(representation) --g--> z, maximize agreement of z; train encoder on imagenet with simCLR, then freeze encoder and train a linear classifier on top with labeled data
Moco: Momentum Contrastive Learning, keep a running queue of keys (negative samples)
DINO: Self-distillation with no labels ; **DINOv2:** auto process web data; multi-objective
contrastive vs. regularized (regularized: no negative samples; need to avoid collapse)
JEPA(Joint Embedding Predictive Architecture) : not generative models or probabilistic models ; representation collapse in JEPA; **LeWorldModel**
SIGReg: minimalist loss (Prediction MSE + SIGReg), prevent mode collapse
Vision-Language Representation Learners: zero shot generalization
CLIP: image-text pairs, pull together similar, push apart different; large scale
SigLIP: decouples batch size **CoCa:** add a decoder with a captioning loss
CLIP-style: issue: caption granularity up to image-level
VLM: **LLaVA:** Large Language and Vision Assistant **Flamingo:** in context learning
Data issue: val data can be leaked; label can be wrong; classes can be ill-defined
L8 Object Detection
 approaches: sliding window (YOLO); region proposal (R-CNN); match patches against codebook (implicit shape model, DETR)
RPN= object vs. non-object(binary classification) + refine box, P(object) & (t_x, t_y, t_w, t_h) ;
anchor: predefined reference bounding boxes, various scale
implicit shape model: extract patches around interest points
mAP@IoU: m: mean over classes; AP: average precision, area under precision-recall curve
 match each pred (sorted by confidence) with the best IoU >= threshold unmatched GT.
 match exists -> TP ++
R-CNN: extract region proposals (~2k) -> each region through convnet -> SVM
Fast R-CNN : propose&crop(maxpool) on the feature map instead of on image; RoIPool
Faster R-CNN: add RPN (region proposal network)
Single-stage object detectors: SSD/RetinaNet/YOLO: output: $(7 \times 7 \times (5 \times B + C))$
 each cell: B base boxes -> final box (dx,dy,dh,dw,confidence); predict scores for each class
DETR: Transformer ; Use learned queries instead of patches extracted from data; no anchor, no NMS; match pred to GT boxes with bipartite matching
open vocabulary: OWL-ViT; LVIS(rare objects dataset); OWLv2; grounding DINO
L9 Dense Prediction
dense prediction task: optical flow, depth estimation, image segmentation, image generation (each pixel's generation is a prediction)
image segmentation: semantic seg: the pixel belongs to cat class ; instance seg: cat1, cat2, ... ; panoptic seg: both background and objects, complete understanding

Semantic Seg: Mean Pixel Classification Acc; Mean per category pixel classification acc; IoU
FCN (Fully Convolutional Network)
DeconvNet:transposed conv;backwards-strided:output stride>1,resolution++, stride=1/2;
max unpooling: remember pooling indices, fill other places with 0
U-Net: fuse by concatenation (skip connections) -> Stable diffusion, U-ViT, PointNet++
Mask R-CNN: Faster R-CNN + FCN on RoIs(regions of interest)
RoIPool: snap to nearest neighbor cell **RoIAlign:** bilinear interpolation
SAM (Segment Anything Model) : model-in-the-loop data engine; interactive seg

$$\text{GAN: } \min_G \max_D (\mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))])$$

D optimal: $\min_G JSD(P_X || P_\theta)$ **WGAN:** JSD -> Wasserstein dist, stable grad, meaningful loss
Pix2PixGAN: conditional mapping ; map input to output by pixel; generator: U-Net;
 discriminator: PatchGAN (dense prediction, patch level instead of img level)

$$\text{DDPM: } x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon$$

Forward Process (Diffusion): $q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t \mathbf{I})$

$$\text{Reverse Process (Denoising): } x_{t-1} = \frac{x_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \epsilon_\theta(x_t, t)}{\sqrt{1 - \beta_t}} + \sigma_t z$$

LDM: diffusion in a compressed latent space learned by a VAE (instead of in pixel space)

DiT: replace U-Net with ViT

L10 Synthetic Data challenge: domain gap (synthetic vs. real);

takeaway: quality > quantity; better inductive bias > more data; paradigm: hybrid training;

task: Point Tracking; 6-DoF Pose Estimation; Depth Estimation; Surface Normal Estimation

L11 curating data for VLM filter text only->filter amplifies bias->scale compensates noise

L12 Image Generation

NCSN: $s_\theta(x) = \nabla_x \log p_\theta(x)$ $x_{k+1} = x_k + \eta s(x_k) + \sqrt{2\eta} z$ (Annealed Langevin Dynamics)

Score SDE: DDPM+NCSN **PF-ODE:** random -> deterministic

L13 Beyond Diffusion

Normalizing flow: $\log p_X(x) = \log p_Z(f(x)) + \log |\det \partial f / \partial x|$

VQ-VAE: image --encoder-> latent z_e --quantize-> codebook z_q --decoder-> image

STE(straight-through estimator): let $\partial L / \partial z_e = \partial L / \partial z_q$ e:codebook entry

$$L = \underbrace{\log p(x | z_q(x))}_{\text{Reconstruction}} + \underbrace{\| \text{sg}[z_e(x)] - e \|^2_2}_{\text{Codebook (VQ)}} + \underbrace{\beta \| z_e(x) - \text{sg}[e] \|^2_2}_{\text{Commitment}}$$

Self-Attention is Four Matrix Multiplies!

Inputs:
 Input vectors: $X [N \times D]$
 Key matrix: $W_K [D \times H_D]$
 Value matrix: $W_V [D \times H_D]$
 Query matrix: $W_Q [D \times H_D]$
 Output matrix: $W_O [H_D \times D]$

Flash Attention algorithm computes 2+3 at the same time without storing the full attention matrix! Makes large N possible

Computation:
 Queries: $Q = XW_Q [H \times N \times D_H]$
 Keys: $K = XW_K [H \times N \times D_H]$
 Values: $V = XW_V [H \times N \times D_H]$
 Similarities: $E = QK^T / \sqrt{d_K} [H \times N \times N]$
 Attention weights: $A = \text{softmax}(E, \text{dim}=2) [H \times N \times N]$
 Head outputs: $Y = AV [H \times N \times D_H] \Rightarrow [N \times H_D]$
 Outputs: $O = YW_O [N \times D]$

QKV Projection
 $[N \times D] [D \times 3H_D] \Rightarrow [N \times 3H_D]$
 Split and reshape to get Q, K, V each of shape $[H \times N \times D_H]$

2. **QK Similarity**
 $[H \times N \times D_H] [H \times D_H \times N] \Rightarrow [H \times N \times N]$

3. **V-Weighting**
 $[H \times N \times N] [H \times N \times D_H] \Rightarrow [H \times N \times D_H]$
 Reshape to $[N \times H_D]$

4. **Output Projection**
 $[N \times H_D] [H_D \times D] \Rightarrow [N \times D]$

Q: How much memory does this take as the number of vectors N increases?
A: $O(N)$ with Flash Attention

